



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

OPTIMIZING COMPUTER SYSTEM PERFORMANCE THROUGH ADVANCED CACHE HIERARCHIES, VIRTUAL MEMORY, AND EMERGING STORAGE TECHNOLOGIES COMPARED WITH TRADITIONAL MEMORY ARCHITECTURES

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA1208 - COMPUTER ARCHITECTURE

to the award of the degree of

BACHELOR OF ENGINEERING / BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Shruthi K. (192571019)

Under the Supervision of

Dr. Rashmita Khilar

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “Optimizing Computer System Performance through Advanced Cache Hierarchies, Virtual Memory, and Emerging Storage Technologies Compared with Traditional Memory Architectures” has been carried out by Shruthi K. (Reg. No. 192571019) under the supervision of Dr. Rashmita Khilar and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Computer Science and Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. Anusaya

Program Director

Department of Computer Science and
Engineering

SIMATS Engineering, SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. Rashmita Khilar

Associate Professor

Department of Computer Science and
Engineering

SIMATS Engineering, SIMATS, Chennai –
602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

I, Shruthi K.(192571019), of the Department of Computer Science and Engineering, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “Optimizing Computer System Performance through Advanced Cache Hierarchies, Virtual Memory, and Emerging Storage Technologies Compared with Traditional Memory Architectures” is the result of my own bonafide efforts. To the best of my knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. I further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. I confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: September 2026

Signature of the Student:

Shruthi K.(192571019)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

(Mandatory for all team projects)

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. %
Shruthi K. / 192571019	End-to-end design, coding, testing and documentation of all three modules (cache hierarchy, virtual memory, storage performance)	Designed cache hierarchy simulator, paging simulator (FIFO/LRU/Optimal), and storage benchmarking dashboard	Executed all test cases, validated hit/miss and page-fault calculations, benchmarked HDD/SATA SSD/NVMe SSD metrics	Authored all chapters (Introduction, Literature Review, Design, Implementation & Results, Conclusion)	100%

Total Contribution: 100%

ABSTRACT

Computer system performance depends significantly on the efficiency of the memory and storage subsystem, yet cache behaviour, virtual-memory paging, and secondary-storage throughput are conventionally analysed in isolation. Traditional single-level memory architectures suffer from high CPU-to-memory access latency, inefficient page replacement, and slow secondary storage, and without a unified way to measure hit ratio, fault rate, latency and throughput together, it is difficult for a designer to know which layer of the memory hierarchy is actually limiting overall system performance.

This project addresses that gap by designing and implementing an integrated, simulation-based performance-analysis system built around three engineering modules. Module 1 models a three-level (L1/L2/L3) cache hierarchy and computes hit ratio and miss ratio for a given stream of memory references. Module 2 implements and compares three classical page-replacement algorithms — First-In-First-Out (FIFO), Least Recently Used (LRU), and Optimal — for virtual-memory paging, tracking page hits, page faults, and fault rate for a configurable number of physical frames. Module 3 benchmarks and compares three classes of secondary storage — traditional Hard Disk Drives (HDD), SATA Solid-State Drives (SSD), and NVMe SSDs — on access latency, sequential transfer speed and normalized throughput/IOPS/efficiency.

The methodology combines analytical modelling (closed-form formulas for hit ratio, fault rate, throughput and latency), discrete-event simulation of memory-reference and page-reference traces, and an interactive dashboard that lets a user vary cache hit ratios, reference strings, frame counts, and replacement policy and immediately visualize the resulting performance.

Simulation results show an overall cache hit ratio of 98.4% (miss ratio 1.6%) is achievable with a well-tuned three-level hierarchy; for a representative reference string with four physical frames, LRU and Optimal both produced 6 page faults against 7 for FIFO, confirming LRU's practical advantage over FIFO with negligible loss relative to the theoretical Optimal bound. On the storage side, NVMe SSD (≈ 5000 MB/s, 0.01–0.05 ms latency) outperformed SATA SSD (≈ 520 MB/s, 0.05–0.2 ms) and HDD (≈ 180 MB/s, 5–10 ms) on every measured metric — throughput, latency, IOPS, parallelism and efficiency.

The project concludes that cache hit ratio, page-replacement efficiency and storage throughput jointly, rather than individually, determine overall system performance, and that a simulation-based, visually-driven comparison is an effective and low-cost way to guide memory-and-storage optimization decisions before committing to hardware.

Keywords

Cache Hierarchy, Virtual Memory, Page Replacement, Hit Ratio, NVMe SSD, Storage Performance Analysis

TABLE OF CONTENTS

- i. CERTIFICATE FROM THE SUPERVISOR
- ii. DECLARATION BY THE CANDIDATE
- iii. INDIVIDUAL CONTRIBUTION STATEMENT
- iv. ABSTRACT
- v. LIST OF FIGURES
- vi. LIST OF TABLES
- vii. LIST OF ABBREVIATIONS
- 1. CHAPTER 1: Introduction and Engineering Problem
- 2. CHAPTER 2: Literature Review and Existing Solutions
- 3. CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security
- 4. CHAPTER 4: Implementation, Testing & Results, Project Management
- 5. CHAPTER 5: Conclusion, Future Work and Reflection
- References
- Appendices
- Capstone Project Outcome Mapping Sheet

LIST OF FIGURES

- Fig. 1.1 System Architecture – Cache, Virtual Memory and Storage Pipeline
- Fig. 3.1 Advanced Cache Hierarchy – Process Flow
- Fig. 3.2 Virtual Memory Management – Process Flow
- Fig. 3.3 Storage Performance Analysis – Process Flow
- Fig. 4.1 Hit Ratio by Cache Level (L1/L2/L3)
- Fig. 4.2 Hit vs Miss Ratio (Donut Chart)
- Fig. 4.3 Page Hit / Page Fault Timeline
- Fig. 4.4 FIFO vs LRU vs Optimal – Page Fault Comparison
- Fig. 4.5 Storage Transfer Speed – HDD vs SATA SSD vs NVMe SSD
- Fig. 4.6 Normalized Storage Performance (Radar Chart)

LIST OF TABLES

- Table 2.1 Comparative Analysis of Existing Approaches
- Table 3.1 Stakeholder / User Requirements
- Table 3.2 Functional & Non-Functional Requirements
- Table 3.3 Constraints & Applicable Standards
- Table 3.4 Design Specifications
- Table 3.5 Alternative Solutions – Evaluation Matrix (Page Replacement)
- Table 3.6 Trade-off Analysis
- Table 3.7 Sustainability, Ethics, Safety, Security
- Table 3.8 Risk Register
- Table 4.1 Tools / Software Used
- Table 4.2 Test Plan & Verification
- Table 4.3 Specification vs Achieved Results
- Table 4.4 Achievement of Objectives
- Table 4.5 Project Milestones & Roles
- Table 4.6 Cost Estimate

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
CPU	Central Processing Unit	—
RAM	Random Access Memory	—
L1/L2/L3	Level 1 / 2 / 3 Cache	—
HDD	Hard Disk Drive	—
SSD	Solid-State Drive	—
NVMe	Non-Volatile Memory Express	—
FIFO	First-In-First-Out (page replacement)	—
LRU	Least Recently Used (page replacement)	—
IOPS	Input/Output Operations Per Second	ops/s
MB/s	Megabytes per Second (throughput)	MB/s
ms / ns	Milliseconds / Nanoseconds (latency)	ms, ns
OS	Operating System	—

CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Modern computer systems rely on a memory hierarchy to bridge the widening speed gap between the CPU and main memory (DRAM). CPU clock speeds have historically improved far faster than DRAM access times, so processors spend a large fraction of cycles waiting for data unless fast, small caches are placed close to the core. An advanced multi-level cache hierarchy — typically L1 (fastest, smallest, per-core), L2 (larger, still per-core or shared between a pair of cores) and L3 (largest, usually shared across all cores) — reduces average access latency by keeping frequently used data close to the CPU. Beyond the cache, the operating system uses virtual memory and paging to let each process address more memory than is physically installed, moving pages between RAM and disk using a page-replacement policy such as FIFO, LRU, or the theoretical Optimal algorithm. Finally, the secondary storage device itself — a traditional HDD, a SATA SSD, or a modern NVMe SSD — determines how quickly pages, files and swapped-out data can be retrieved. Each of these three layers is well studied individually, but they are rarely evaluated together using a common, measurable set of metrics.

1.2 Need for the Project

Engineers building or configuring a computer system frequently need to decide where to invest limited resources — a bigger cache, a smarter page-replacement policy, or a faster storage device — to get the best performance return. Existing academic and industrial studies typically isolate a single layer (e.g., a new cache-replacement policy evaluated only on cache hit rate, or a storage benchmark evaluated only on throughput), which makes it hard for a systems designer, a student, or a hardware buyer to reason about the combined effect of cache, paging and storage decisions on end-to-end performance. Users affected include systems architects selecting hardware configurations, students learning computer-architecture concepts, and organisations sizing infrastructure for memory- or I/O-bound workloads. The engineering significance lies in providing one simulation environment where the interaction between these three layers can be observed directly, rather than inferred from separate, disconnected studies.

1.3 Problem Statement

Traditional memory and storage systems are often evaluated in isolation, without a unified view of cache, virtual memory, and storage performance. This makes it difficult to identify which layer — cache hierarchy, paging policy, or storage device — most limits system performance. A simulation-based system is therefore required to model L1/L2/L3 cache behaviour, page-replacement policies, and storage throughput, and to compare them using measurable metrics such as hit ratio, fault rate, latency, and throughput. The problem is engineering-complex because it involves multiple interacting factors (cache configuration, workload access pattern, replacement policy, physical frame count, and storage device class), conflicting requirements (higher hit ratio generally requires more cache area and cost; lower fault rate requires more physical frames and hence more RAM; higher storage throughput requires more expensive NVMe hardware), and realistic constraints (limited cache/frame budget, need for real-time visual feedback, and the requirement that the simulation remain accurate against known analytical formulas).

1.4 Project Aim

To design, implement, and evaluate an integrated simulation system that models an advanced cache hierarchy, virtual-memory page-replacement policies, and secondary-storage performance, so that the layer most limiting overall computer-system performance can be identified and optimization strategies can be recommended.

1.5 Project Objectives

- To study the L1, L2, and L3 cache hierarchy and analyze hit ratio and miss ratio.
- To implement and compare FIFO, LRU, and Optimal page-replacement algorithms.
- To evaluate HDD, SATA SSD, and NVMe SSD performance in terms of latency and throughput.
- To simulate cache and virtual-memory behaviour and visualize performance graphically.
- To recommend optimization strategies for building faster and more efficient computer systems.

1.6 Scope

Included: simulation and analytical modelling of a 3-level cache hierarchy, simulation of FIFO/LRU/Optimal page replacement over user-supplied reference strings, and comparative benchmarking of HDD, SATA SSD and NVMe SSD using representative, published latency/throughput figures; an interactive dashboard for varying inputs and visualizing hit ratio, fault rate, transfer speed and normalized performance.

Excluded: physical fabrication of cache/memory hardware, live benchmarking of a specific physical machine's cache/disk, and cache-coherence/multi-core concurrency effects.

Assumptions: memory-reference and page-reference streams used are representative synthetic traces; storage figures are representative manufacturer-class values for HDD, SATA SSD and NVMe SSD rather than measurements from one specific model.

Boundary conditions: cache hierarchy limited to three levels (L1/L2/L3); page-replacement frame count is user-configurable but bounded by the simulator's input range; storage comparison limited to the three device classes named above.

1.7 Expected Engineering Outcome

The expected outcome is a working simulation model / interactive dashboard (software prototype) comprising three linked modules — an Advanced Cache Hierarchy simulator, a Virtual Memory (paging) simulator, and a Storage Performance Analysis comparator — that together compute and visualize hit ratio, miss ratio, page faults, fault rate, latency and throughput, and that produce a consolidated performance report and set of optimization recommendations.

CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature was identified by searching for recent (2021–2023) peer-reviewed work on cache-replacement policies, page-table/tiered-memory management, memory-allocation contiguity, and persistent/emerging memory characterization, using computer-architecture venues (ACM SIGARCH / ISCA-adjacent publications) as the primary source. Five representative papers were selected to cover the three layers addressed by this project: cache replacement, virtual-memory/page-table management, and storage/persistent-memory characterization.

2.2 Review of Existing Work

Sethumurugan et al. (2021) developed a low-overhead, machine-learning-based replacement policy for shared last-level caches, training a lightweight predictor on eviction outcomes and validating it in simulation against CRC2 competition baselines across a multi-core cache hierarchy. Yang et al. (2023) proposed the Prefetch-Adaptive Intelligent Cache replacement policy (PAIC), which builds separate ML predictor tables for prefetched versus demand accesses and evaluates the approach on SPEC2006/2017 benchmarks. On the virtual-memory side, Kumar et al. (2021) introduced “Radiant,” a page-table placement scheme for hybrid fast/slow tiered-memory systems, restructuring page-table lookup and placement and evaluating it through system-level simulation. Skarlatos et al. (2023) investigated contiguity-aware physical-memory allocation (“Contiguitas”) to reduce address-translation overhead in large-memory datacenter workloads. Finally, Xiang et al. (2022) performed an in-depth microbenchmarking study characterizing the latency and bandwidth of Intel Optane DC persistent memory under varied read/write patterns and access granularities.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
ML-based LLC replacement (Sethumurugan et al., 2021)	Improves hit ratio over static heuristics; low run-time overhead	Needs per-workload retraining; added hardware overhead limits low-power use	Motivates modelling hit/miss ratio as the core cache metric in Module 1
Prefetch-adaptive replacement, PAIC (Yang et al., 2023)	Separately optimizes prefetch vs. demand accesses	Extra predictor storage increases complexity; gains vary by benchmark	Confirms that access-pattern-aware policies materially change hit ratio, as simulated here
Radiant page-table placement (Kumar et al., 2021)	Faster page-table lookup on tiered fast/slow memory	Tailored to tiered hardware; adds OS-level complexity; single-tier DRAM only	Informs the paging/page-table model used in Module 2
Contiguitas allocation (Skarlatos et al., 2023)	Cuts address-translation overhead via contiguity-aware allocation	Depends on allocation pattern; needs allocator/OS changes	Reinforces that page-replacement efficiency (Module 2) is a first-order performance lever
Optane persistent-memory characterization (Xiang et al., 2022)	Detailed empirical latency/bandwidth data for emerging memory	Tied to now-discontinued hardware; read/write latency asymmetry persists	Provides the methodological basis for the HDD/SATA-SSD/NVMe-SSD benchmarking in Module 3

2.4 Research / Engineering Gap

All five studies optimize a single layer of the memory/storage stack in isolation — cache replacement, page-table placement, or storage-device characterization — and none combine cache hit ratio, page-fault rate, and storage

throughput into one measurable, side-by-side comparison. Consequently there is no lightweight tool that lets a designer see, for the same simulated workload, which of the three layers is the actual bottleneck.

2.5 Proposed Contribution

This project contributes an integrated three-module simulator that (i) reuses the well-established hit-ratio/miss-ratio formulation from cache-replacement literature, (ii) reuses the classical FIFO/LRU/Optimal page-replacement formulation from virtual-memory literature, and (iii) reuses representative latency/throughput figures from storage-characterization literature, and combines all three into a single interactive dashboard with consolidated visual analytics — something the reviewed individual studies do not provide.

CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Systems designer / student	Needs to see hit ratio, fault rate, latency and throughput for a given configuration in one dashboard
Course evaluator (SIMATS Engineering)	Needs the simulation to follow correct, verifiable analytical formulas
End user of the dashboard	Needs to vary hit ratios, reference strings, frame counts, and algorithm choice and see results instantly

Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Compute L1/L2/L3 hit ratio and overall miss ratio from configurable hit-ratio sliders	Result matches formula $H = \Sigma \text{Hits} / \text{Accesses}$ to within 0.1%
Functional	Simulate FIFO, LRU and Optimal page replacement on a reference string	Correct frame contents at every step, verified by hand-trace
Functional	Compare HDD, SATA SSD, NVMe SSD on latency and throughput	Displays all three devices side-by-side with numeric values
Non-Functional (Performance)	Dashboard updates results after “Run Simulation” within perceptible real time	< 1 second for inputs up to 200 references
Non-Functional (Usability)	Provide graphical (bar/donut/radar) visualization of every result set	At least one chart per module

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
IEEE 610.12 (Software Engineering Terminology)	Consistent definition of engineering terms (hit ratio, fault rate, throughput)	Terminology and formulas in this report follow IEEE definitions
IEEE citation style	Consistent, traceable referencing	All references in Chapter References follow IEEE numbering/format

3.3 Design Specifications

Parameter	Target/Requirement	Unit	Priority
L1 cache hit ratio (typical)	≥ 85	%	High
L2 cache hit ratio (typical)	40 – 60	%	Medium
L3 cache hit ratio (typical)	55 – 70	%	Medium
Overall memory-access hit ratio	≥ 95	%	High
Page-fault rate (4 frames, sample workload)	≤ 50	%	High
NVMe SSD throughput	≥ 3000	MB/s	High
NVMe SSD latency	≤ 0.05	ms	High

3.4 Alternative Solutions & Evaluation

Alternative 1 (FIFO): Evicts the page that has been resident longest, regardless of recent use — simple to implement (a queue) but ignores locality of reference.

Alternative 2 (LRU): Evicts the page that has not been used for the longest time, approximating temporal locality — more accurate than FIFO but requires tracking recency of every frame.

Alternative 3 (Optimal / Belady's algorithm): Evicts the page that will not be used for the longest time in the future — provably the best possible, but requires future knowledge and is used only as a theoretical benchmark.

Criterion	Weight	FIFO	LRU	Optimal
Page-fault rate (lower is better)	0.4	Highest faults (7)	Low faults (6)	Lowest possible (6, theoretical bound)
Implementation cost	0.2	Very low (queue)	Medium (recency tracking)	Not implementable in real time
Practicality for real OS use	0.3	High	High	None (offline only)
Educational/benchmark value	0.1	Medium	High	High (upper bound reference)

3.5 Selected Design & Detailed Design

LRU was selected as the recommended real-world policy because it matched the Optimal algorithm's page-fault count (6 faults) on the representative reference string while remaining implementable without future knowledge, whereas FIFO produced one additional fault (7); this is evidence-based selection from the matrix in Section 3.4, not a preference.

System architecture: user/process workload enters Module 1 (Advanced Cache Hierarchy), which checks L1, then L2 on an L1 miss, then the shared L3 on an L2 miss, and finally main memory on an L3 miss, computing hit ratio and miss ratio and analyzing latency at each stage. In parallel, Module 2 (Virtual Memory Management) takes a CPU-generated virtual address, checks the page table, and on a page hit accesses the physical frame directly; on a page fault it loads the page from disk and applies the selected FIFO/LRU/Optimal replacement policy before updating the page table. Module 3 (Storage Performance Analysis) benchmarks the selected storage class (HDD / SATA SSD / NVMe SSD) for access time, data-transfer speed and throughput. A final Performance Analysis & Output stage compares cache hit ratio, page-replacement efficiency and storage class across all three modules and produces an overall performance report.

Key engineering calculations:

- L1 Hit Ratio (%) = (L1 Hits / L1 Accesses) × 100; similarly for L2 and L3, where Accesses at a level = requests that reach that level.
- Total Hits (H) = L1 Hits + L2 Hits + L3 Hits; Total Misses (M) = Memory Accesses – Total Hits (i.e., L3 misses that go to main memory).
- Overall Hit Ratio (%) = (Total Hits / Memory Accesses) × 100; Miss Ratio (%) = 100 – Hit Ratio (%).
- Page Hits (H) = Total References (N) – Page Faults (F_fault); Hit Rate (%) = (H / N) × 100; Fault Rate (%) = (F_fault / N) × 100 = 100 – Hit Rate (%).
- Storage Throughput (MB/s) = Total Data Transferred (MB) / Total Time Taken (s); Latency (ms) = Total Time Taken (ms) / Number of Operations.
- Normalized Performance (for radar chart) = Actual Value of metric M / Maximum value of M among {HDD, SATA SSD, NVMe SSD}, giving a 0–1 score per metric (latency, throughput, IOPS, parallelism, efficiency).

Systems approach: the three modules interface through a common “memory access request” abstraction — a request that misses at every cache level becomes a main-memory/virtual-memory request, and a virtual-memory page fault becomes a storage-device read request — so a change in one layer (e.g., a lower cache hit ratio) directly increases load on the next layer down (page table, then storage), which is why the Performance Analysis stage reports all three together rather than independently.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Page-replacement policy	FIFO vs LRU vs Optimal	LRU: near-optimal fault rate; FIFO: simplest to implement	LRU needs recency tracking; Optimal is non-implementable online	LRU chosen — matched Optimal's 6 faults vs FIFO's 7 (Section 3.4)
Cache hierarchy depth	2-level vs 3-level	3-level (L1/L2/L3) captures realistic modern CPU behaviour	More levels add simulation and explanation complexity	3-level retained to match real-world CPU architectures
Storage benchmark scope	Single device vs 3-device comparison	3-device (HDD/SATA/NVMe) comparison shows relative gains	Requires representative rather than lab-measured figures	3-device comparison retained — directly answers Objective 3

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Faster storage (NVMe) reduces active I/O wait time and hence energy drawn per completed operation	Manufacturer-class NVMe throughput (≈ 5000 MB/s) vs HDD (≈ 180 MB/s) at comparable idle power	Recommendation in Chapter 5 favours NVMe/SSD migration for I/O-bound workloads to cut energy-per-operation
Ethics	Simulation results must not be presented as measurements from real hardware	Representative published latency/throughput ranges were used, clearly labelled as such	All storage figures in this report are explicitly identified as representative, not lab-measured
Health & Safety	Not physically applicable (software-only simulation); no hardware handling involved	N/A — no physical prototype was built	No safety mitigation required for this deliverable
Security	A simulator that mimics real cache/paging behaviour could, in principle, be misused to model a timing side-channel	Reviewed for any executable code paths that measure real hardware timing; none present — all timings are simulated constants	Confirmed the tool operates only on synthetic data and cannot be used to fingerprint real hardware

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Simulation formulas diverge from textbook definitions	Low	High	Medium	Cross-checked every formula against Hennessy & Patterson (2019) and Jacob et al. (2010)	Low
Representative storage figures misread as real benchmarks	Medium	Medium	Medium	Explicitly labelled all storage numbers as representative/typical values	Low
Single-student project timeline slippage	Medium	Medium	Medium	Modular design allowed each module to be built and tested independently	Low

CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed iteratively, module by module: Module 1 (cache hierarchy) was built and validated first using known hit/miss traces, Module 2 (virtual memory) was then built and cross-checked against hand-worked FIFO/LRU/Optimal traces, and Module 3 (storage performance analysis) was built last using representative published latency/throughput figures for HDD, SATA SSD and NVMe SSD. Each module exposes an interactive dashboard (sliders/inputs for hit ratios, reference strings, frame counts and algorithm selection) with a “Run Simulation” action that computes results and renders charts.

Resource / Material	Purpose
Synthetic memory-access traces	Input workload for the cache-hierarchy simulator
Sample page-reference string (7,0,1,2,0,3,0,4,2,3,0,3,2)	Input workload for the paging simulator
Representative HDD / SATA SSD / NVMe SSD specifications	Input dataset for the storage-performance comparator
Charting library (bar / donut / radar)	Graphical visualization of results

4.2 Software Implementation & Modern Tools Used

Module 1 — Advanced Cache Hierarchy: implements the CPU-request → L1 → L2 → L3 → main-memory lookup chain described in Section 3.5, and computes per-level and overall hit/miss ratios and access latency for each simulated request.

Module 2 — Virtual Memory Management: implements a paging simulator that accepts a reference string and a frame count, then applies the selected replacement algorithm (FIFO, LRU or Optimal), producing a page-by-page hit/fault trace and the final frame contents.

Module 3 — Storage Performance Analysis: implements a comparator that takes representative access-time and throughput figures for HDD, SATA SSD and NVMe SSD and computes normalized (0–1) scores per metric for the radar-chart visualization.

Tool / Software	Purpose	Stage Used
Interactive web dashboard (cache/paging/storage simulator)	Runs simulations and renders bar/donut/radar charts	Implementation & Results
Spreadsheet-style calculators for hit-ratio / fault-rate formulas	Cross-validates simulator output against Section 3.5 formulas	Testing & Verification

4.3 Test Plan & Verification

Requirement	Test Method		Acceptance Criterion
Cache hit/miss ratio correctness	Hand-trace 10 memory accesses against simulator output		Simulator matches hand-trace exactly
Page replacement correctness (FIFO/LRU/Optimal)	Hand-trace reference string 7,0,1,2,0,3,0,4,2,3,0,3,2 with 4 frames		Simulator's fault count matches hand-trace for each algorithm
Storage comparison correctness	Recompute throughput/latency normalization by hand for all three devices		Normalized values match to 2 decimal places
Specification	Required Value	Achieved Value	Status
Overall cache hit ratio	≥ 95%	98.4%	Pass
L1 hit ratio	≥ 85%	92%	Pass
Page-fault rate (LRU, 4 frames)	≤ 50%	46.2%	Pass
NVMe SSD throughput	≥ 3000 MB/s	5000 MB/s	Pass
NVMe SSD latency	≤ 0.05 ms	0.01–0.05 ms	Pass

4.4 Results

Module 1 (100 simulated memory accesses, L1/L2/L3 hit ratios of 92%/47%/62% respectively): overall Hit Ratio = 98.4%, Miss Ratio = 1.6%, with 98 total hits (92 at L1, 4 at L2, 3 at L3 after cascading misses) against 2 total misses reaching main memory. A donut chart shows the overwhelming majority of requests are satisfied at L1 alone, and a bar chart shows hit ratio declining from L1 to L2 before recovering somewhat at the shared L3.

Module 2 (reference string 7,0,1,2,0,3,0,4,2,3,0,3,2 with 4 physical frames): LRU produced 7 page hits and 6 page faults (fault rate 46.2%), with final frame contents {3, 0, 4, 2}. Comparing all three policies on the same reference string: FIFO produced 7 faults, LRU produced 6 faults, and Optimal produced 6 faults — confirming LRU reaches the theoretical Optimal bound on this workload while FIFO does not.

Module 3 (representative device figures): HDD ≈ 180 MB/s throughput at 5–10 ms latency; SATA SSD ≈ 520 MB/s at 0.05–0.2 ms; NVMe SSD ≈ 5000 MB/s at 0.01–0.05 ms. The transfer-speed bar chart shows NVMe SSD roughly 9.6× the throughput of SATA SSD and about 28× that of HDD; the normalized radar chart shows NVMe SSD leading on every axis — latency, throughput, IOPS, parallelism and efficiency.

4.5 Data Analysis & Engineering Judgment

The gap between L1 (92%) and L2 (47%) hit ratios in Module 1 is expected: L2 only ever sees requests that already missed in L1, so its hit ratio reflects a harder, pre-filtered access pattern rather than a design flaw — the metric that matters for overall performance is the cascaded Hit Ratio (98.4%), not any single level in isolation. In Module 2, LRU equalling the Optimal fault count (6) rather than merely improving on FIFO indicates the chosen reference string has strong short-term temporal locality, which LRU is specifically designed to exploit; a workload with weaker locality would be expected to show a larger gap between LRU and Optimal. In Module 3, the roughly order-of-magnitude throughput gains of NVMe over SATA SSD and two-orders-of-magnitude gain over HDD are consistent with the fundamentally different interconnects (PCIe/NVMe protocol vs. SATA) rather than incremental engineering, which is why the recommendation in Section 4.7 favours a storage-class change over further cache tuning for I/O-bound workloads.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Study L1/L2/L3 hierarchy; analyze hit/miss ratio	98.4% overall hit ratio computed and visualized (Section 4.4)	Fully Achieved
Implement & compare FIFO, LRU, Optimal	7 vs 6 vs 6 page faults on identical reference string (Section 4.4)	Fully Achieved
Evaluate HDD, SATA SSD, NVMe SSD latency/throughput	180/520/5000 MB/s and 5–10/0.05–0.2/0.01–0.05 ms compared (Section 4.4)	Fully Achieved
Simulate and visualize performance graphically	Bar, donut and radar charts produced for all three modules	Fully Achieved
Recommend optimization strategies	See Sections 4.7 and 5.1–5.2	Fully Achieved

4.7 Strengths & Limitations

Strengths:

- Combines cache, virtual-memory and storage analysis in one consistent dashboard, unlike the single-layer studies reviewed in Chapter 2.
- All formulas are transparent and independently hand-verifiable (Section 3.5).
- Interactive sliders/inputs let a user re-run the simulation instantly for different workloads.

Limitations:

- Storage figures are representative manufacturer-class values, not measurements from one physical device, and should not be quoted as guaranteed real-world performance.
- The cache and paging simulators use synthetic access/reference strings rather than traces captured from a real running application.
- Multi-core cache-coherence effects and concurrent access patterns are outside the scope of this simulation.

4.8 Project Planning, Roles & Budget

Milestone	Target Week	Status
Literature review and problem definition	Week 1–2	Completed
Module 1 (Cache Hierarchy) design & build	Week 3–4	Completed
Module 2 (Virtual Memory) design & build	Week 5–6	Completed
Module 3 (Storage Performance) design & build	Week 7	Completed
Integration, testing & report writing	Week 8–9	Completed

Student	Assigned Responsibility	Actual Contribution
Shruthi K. (192571019)	All three modules, testing, and report	100%

Item	Estimated Cost	Actual Cost
Software/tools (dashboard, charting library)	No additional cost (open-source/in-browser)	₹0
Compute resources for simulation	Standard laptop/desktop	₹0 (existing hardware)

CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This project successfully integrates an advanced cache hierarchy, virtual-memory management, and storage-performance analysis to analyze computer-system performance within a single framework. The cache module demonstrates how L1, L2, and L3 hit/miss analysis improves the overall hit ratio and reduces effective access latency, achieving a 98.4% overall hit ratio in simulation. The virtual-memory module compares FIFO, LRU, and Optimal page replacement using page-hit and page-fault analysis, showing that LRU (6 faults) matches the theoretical Optimal bound and outperforms FIFO (7 faults) on a representative workload. The storage module compares HDD, SATA SSD, and NVMe SSD using access time, transfer speed, and throughput, confirming that NVMe SSD (≈ 5000 MB/s, 0.01–0.05 ms) decisively outperforms SATA SSD (≈ 520 MB/s) and HDD (≈ 180 MB/s) on every metric. Overall, the implemented system provides a practical, low-cost way to analyze and compare cache, memory, and storage performance for building faster computer systems, and directly satisfies every objective set out in Chapter 1.

5.2 Future Work

- **AI-Based Cache Optimization:** use machine learning to predict cache hit and miss patterns and further improve the hit ratio, building on the ML-based replacement policies reviewed in Chapter 2.
- **Adaptive Page Replacement:** combine FIFO, LRU, and Optimal strategies into an adaptive policy that reduces the page-fault rate further than any single fixed policy.
- **Next-Generation Storage:** extend the throughput and latency analysis to newer NVMe generations and persistent-memory devices (e.g., Optane-class technology).
- **Heterogeneous Architecture Testing:** evaluate the system across different CPU, memory, and storage architectures to compare performance under varied workloads.
- **Real-Time Performance Analysis:** enhance the simulator with real-time monitoring, advanced benchmarking, and continuous graphical performance analytics captured from an actual running system rather than synthetic traces.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired:

- Practical application of hit-ratio, fault-rate, throughput and latency formulas to a working simulation rather than only textbook exercises.
- Comparative behaviour of FIFO, LRU and Optimal page-replacement policies on a concrete reference string.
- Order-of-magnitude performance differences between HDD, SATA SSD and NVMe SSD storage classes.

Engineering & Professional Skills Developed:

- Structuring an engineering report against a formal rubric (requirements, trade-offs, risk register, verification).
- Translating architectural diagrams into working simulation logic.
- Presenting quantitative results using appropriate charts (bar, donut, radar) rather than raw numbers alone.

Individual Reflection — Shruthi K.:

The most difficult engineering problem encountered was ensuring the three modules — cache, virtual memory, and storage — used a consistent set of formulas and terminology so their results could be meaningfully compared side-

by-side; this was solved by deriving every metric from the same base definitions (hits, misses, accesses, time) documented in Section 3.5. A key engineering decision made personally was selecting LRU as the recommended page-replacement policy, based directly on the evaluation matrix in Section 3.4 rather than general reputation, after confirming it matched the Optimal algorithm's fault count on the test workload. Independently, I learned how much the choice of secondary-storage class (HDD vs. SSD vs. NVMe) can dominate overall system performance compared to cache or paging tuning alone. If repeated, I would redesign the project to additionally capture memory-access traces from a real running application rather than relying solely on synthetic reference strings, to validate the simulator against real-world workload behaviour.

REFERENCES

- [1] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Burlington, MA, USA: Morgan Kaufmann, 2019.
- [2] B. Jacob, S. Ng, and D. Wang, Memory Systems: Cache, DRAM, Disk. Burlington, MA, USA: Morgan Kaufmann, 2010.
- [3] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely, and J. Emer, "Adaptive insertion policies for high-performance caching," ACM SIGARCH Comput. Archit. News, vol. 35, no. 2, pp. 381–391, 2007.
- [4] A. Basu, J. Gandhi, J. Chang, M. D. Hill, and M. M. Swift, "Efficient virtual memory for big memory servers," ACM SIGARCH Comput. Archit. News, vol. 41, no. 3, pp. 237–248, 2013.
- [5] S. Mittal and J. S. Vetter, "A survey of software techniques for using non-volatile memories for storage and main memory systems," IEEE Trans. Parallel Distrib. Syst., vol. 27, no. 5, pp. 1537–1550, 2016.

APPENDICES

Appendix A – Detailed Calculations — worked hand-traces for hit-ratio, fault-rate and throughput formulas underlying Section 3.5.

Appendix B – Engineering Drawings / Schematics — system-architecture and module-flow diagrams referenced in Figures 1.1, 3.1–3.3.

Appendix C – Source Code / Code Repository Information — complete simulator source is maintained in the project repository; only essential logic is described in Chapter 4.

Appendix D – Datasheets — representative HDD / SATA SSD / NVMe SSD specification ranges used in Module 3.

Appendix E – Experimental / Raw Data — the sample memory-access trace and reference string (7,0,1,2,0,3,0,4,2,3,0,3,2) used for Modules 1 and 2.

Appendix F – Ethics / Institutional Approval — not applicable; no human subjects or physical hardware testing involved.

Appendix G – Project Meeting / Progress Records — milestone table in Section 4.8.

Appendix H – User/Industry Feedback — not collected for this iteration; noted as future work.

Appendix I – Publications / Patents / Competitions / Product Outcomes — none at this stage.

Appendix J – Individual Contribution Evidence — see Individual Contribution Statement (front matter).

CAPSTONE PROJECT OUTCOME MAPPING SHEET

(To be completed by the department/faculty assessor.)

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)